

Experiment: Build an IoT Project Using an Online IoT Platform

Aim

To build an IoT project using an online IoT platform such as ThingsBoard or Thinger.io and monitor sensor data remotely over the internet.

Apparatus Required

- ESP32 or Raspberry Pi
- DHT11/DHT22 sensor
- Breadboard and jumper wires
- USB cable
- Wi-Fi network
- Computer with Arduino IDE or Thonny IDE
- Online IoT platform account:
 - ThingsBoard
 - Thinger.io

Theory

Internet of Things (IoT) platforms allow devices to connect to the cloud and exchange data remotely. These platforms provide: Device management Real-time monitoring Data visualization Dashboard creation Remote control of devices In this experiment: An ESP32 or Raspberry Pi collects sensor data. The device connects to the internet through Wi-Fi. Sensor data is sent to an online IoT platform. The platform displays the data on dashboards and graphs. ThingsBoard and Thinger.io are popular cloud-based IoT platforms used for remote monitoring and IoT application development.

Procedure

1. Create an account on ThingsBoard or Thinger.io.
2. Create a new IoT device on the platform.
3. Note the device credentials or access token.
4. Connect the DHT11 sensor to the ESP32 or Raspberry Pi.
5. Connect the IoT device to a Wi-Fi network.
6. Write the program to read sensor data.
7. Add the platform credentials in the program.
8. Upload and run the program.
9. Open the online dashboard.
10. Observe real-time temperature and humidity data on the platform.

Working Principle

The ESP32 reads temperature and humidity values from the DHT sensor. The ESP32 connects to the Wi-Fi network. Sensor data is sent to the IoT cloud platform using HTTP requests. The cloud platform stores and visualizes the data. Users can monitor sensor readings remotely through dashboards.

Applications

Smart home systems Weather monitoring Industrial automation Smart agriculture Remote environmental monitoring
IoT-based analytics systems

Conclusion

The experiment was successfully performed, and an IoT project was built using an online IoT platform. Sensor data from the ESP32 was transmitted to the cloud platform and displayed remotely on dashboards, demonstrating IoT connectivity, cloud monitoring, and real-time data visualization.

Sample Arduino Code (ESP32)

```
#include <WiFi.h>
#include <HTTPClient.h>
#include "DHT.h"

#define DHTPIN 4
#define DHTTYPE DHT11

const char* ssid = "Your_WiFi_Name";
const char* password = "Your_WiFi_Password";

String serverName = "https://demo.thingsboard.io/api/v1/YOUR_TOKEN/telemetry";

DHT dht(DHTPIN, DHTTYPE);

void setup() {
    Serial.begin(115200);

    WiFi.begin(ssid, password);

    while (WiFi.status() != WL_CONNECTED) {
        delay(1000);
    }

    dht.begin();
}

void loop() {
    float temp = dht.readTemperature();
    float hum = dht.readHumidity();

    if(WiFi.status() == WL_CONNECTED) {
        HTTPClient http;

        http.begin(serverName);
        http.addHeader("Content-Type", "application/json");

        String data = "{\"temperature\": " + String(temp) +
            ", \"humidity\": " + String(hum) + "}";

        http.POST(data);

        http.end();
    }

    delay(5000);
}
```

Experiment: IoT on Google Cloud

Aim

To connect an IoT device to Google Cloud and send sensor data for cloud-based monitoring and processing.

Apparatus Required

- ESP32 or Raspberry Pi
- DHT11/DHT22 sensor (optional)
- Breadboard and jumper wires
- USB cable
- Wi-Fi network
- Google Cloud account
- Computer with Arduino IDE or Thonny IDE

Theory

Internet of Things (IoT) devices can collect data from sensors and send it to cloud platforms for storage, analysis, and remote monitoring. Google Cloud provides cloud services for IoT applications such as: Data collection Device management Cloud storage Data analytics Remote monitoring In this experiment: The IoT device connects to the internet using Wi-Fi. Sensor data is generated or collected. Data is transmitted to Google Cloud using HTTP or MQTT protocols. Google Cloud stores and processes the received data. Cloud-based IoT systems are widely used in smart homes, industries, healthcare, and agriculture.

Procedure

1. Create a Google Cloud account.
2. Create a new Google Cloud project.
3. Enable IoT and cloud services.
4. Connect the ESP32 or Raspberry Pi to a Wi-Fi network.
5. Write the program to collect sensor data.
6. Configure cloud credentials and API details.
7. Upload the code to the IoT device.
8. Run the program.
9. Observe sensor data being transmitted to Google Cloud.

Working Principle

The IoT device connects to the internet through Wi-Fi. Sensor data is collected periodically. The device sends data to Google Cloud using APIs or MQTT. Google Cloud receives, stores, and processes the data. Users can monitor the data remotely through cloud dashboards and applications.

Applications

Smart home automation Weather monitoring systems Industrial IoT Smart agriculture Healthcare monitoring Remote asset tracking

Conclusion

The experiment was successfully performed, and IoT communication with Google Cloud was established. Sensor data was transmitted from the IoT device to the cloud platform, demonstrating cloud connectivity, remote monitoring, and IoT data management using Google Cloud.

Sample Python Code (Raspberry Pi)

```
import requests
import json
import random
import time

url = "https://example.googleapis.com/data"

while True:

    temperature = random.randint(20, 35)
    humidity = random.randint(40, 70)

    data = {
        "temperature": temperature,
        "humidity": humidity
    }

    response = requests.post(url, json=data)

    print("Data Sent:", data)
    print("Response:", response.status_code)

    time.sleep(5)
```

Experiment: MicroPython MQTT Weather Logger using ESP32

Aim

To develop a weather logging system using ESP32 and MicroPython that publishes temperature and humidity data to an MQTT broker.

Apparatus Required

- ESP32
- DHT11 or DHT22 temperature and humidity sensor
- Breadboard
- Jumper wires
- USB cable
- Computer with Thonny IDE
- Wi-Fi network
- MQTT broker (Mosquitto or HiveMQ)

Theory

MQTT (Message Queuing Telemetry Transport) is a lightweight communication protocol commonly used in IoT applications. It follows a publish-subscribe model where devices publish sensor data to an MQTT broker, and other devices can subscribe to receive the data.

In this experiment: The ESP32 reads temperature and humidity values from the DHT sensor. The ESP32 connects to a Wi-Fi network. Sensor data is published to an MQTT broker using MicroPython. The MQTT broker stores and distributes the data to subscribed devices. This system can be used for wireless weather monitoring and IoT applications.

Procedure

1. Connect the DHT11 sensor VCC pin to the 3.3V pin of ESP32.
2. Connect the GND pin of the sensor to the ESP32 GND.
3. Connect the DATA pin of the sensor to GPIO pin 4 of ESP32.
4. Install MicroPython firmware on the ESP32.
5. Open Thonny IDE and connect to the ESP32.
6. Install the MQTT library (umqtt.simple) if required.
7. Write the MicroPython code for Wi-Fi and MQTT communication.
8. Enter Wi-Fi and MQTT broker credentials in the program.
9. Run the program.
10. Observe temperature and humidity data being published to the MQTT broker.

Working Principle

The ESP32 connects to the Wi-Fi network. The DHT sensor measures temperature and humidity. The ESP32 reads the sensor values using MicroPython. Sensor data is published to the MQTT broker. Other IoT devices or applications subscribed to the MQTT topic can receive the weather data wirelessly.

Applications

Smart weather monitoring IoT environmental monitoring Smart agriculture Home automation Industrial monitoring systems Remote sensing applications

Conclusion

The experiment was successfully performed, and the ESP32 established MQTT communication using MicroPython. Temperature and humidity data from the DHT sensor were published wirelessly to the MQTT broker, demonstrating IoT communication and real-time weather logging using ESP32.

MicroPython Code

```
import network
import time
from machine import Pin
import dht
from umqtt.simple import MQTTClient

# Wi-Fi credentials
SSID = "Your_WiFi_Name"
PASSWORD = "Your_WiFi_Password"

# MQTT Broker details
MQTT_BROKER = "broker.hivemq.com"
CLIENT_ID = "ESP32_Weather"
TOPIC = b"weather/data"

# Connect to Wi-Fi
wifi = network.WLAN(network.STA_IF)
wifi.active(True)
wifi.connect(SSID, PASSWORD)

while not wifi.isconnected():
    pass

print("Connected to Wi-Fi")

# Initialize DHT Sensor
sensor = dht.DHT11(Pin(4))

# Connect to MQTT Broker
client = MQTTClient(CLIENT_ID, MQTT_BROKER)
client.connect()

print("Connected to MQTT Broker")

while True:
    sensor.measure()

    temp = sensor.temperature()
    hum = sensor.humidity()

    message = "Temp: {}C Humidity: {}%".format(temp, hum)

    client.publish(TOPIC, message)

    print("Published:", message)

    time.sleep(5)
```

Experiment: Establish Bluetooth Communication Between a Raspberry Pi Pico

Aim

To establish Bluetooth communication using a Raspberry Pi Pico and exchange data wirelessly with another Bluetooth-enabled device.

Apparatus Required

- Raspberry Pi Pico W
- USB cable
- Computer or smartphone with Bluetooth support
- MicroPython firmware installed on Raspberry Pi Pico W
- Thonny IDE or any Python IDE

Theory

Bluetooth is a wireless communication technology used for short-range data exchange between devices. The Raspberry Pi Pico W supports wireless communication and can be programmed to communicate with nearby Bluetooth-enabled devices.

In this experiment: The Raspberry Pi Pico W initializes Bluetooth communication. A nearby device such as a smartphone or laptop connects to the Pico W. Data is exchanged wirelessly through Bluetooth communication. Bluetooth communication is commonly used in IoT systems, wireless sensors, robotics, and smart devices.

Procedure

1. Connect the Raspberry Pi Pico W to the computer using a USB cable.
2. Install MicroPython firmware on the Pico W.
3. Open Thonny IDE on the computer.
4. Write the Bluetooth communication program.
5. Save the program to the Raspberry Pi Pico W.
6. Run the program.
7. Enable Bluetooth on the smartphone or laptop.
8. Pair the device with the Raspberry Pi Pico W.
9. Exchange data wirelessly and observe successful communication.

Working Principle

The Raspberry Pi Pico W activates its Bluetooth module. Nearby Bluetooth devices detect the Pico W. A wireless Bluetooth connection is established. Data can be transmitted and received wirelessly between connected devices.

Applications

IoT devices Wireless sensor networks Smart home systems Robotics Wireless data transfer Mobile device communication

Conclusion

The experiment was successfully performed, and Bluetooth communication was established using the Raspberry Pi Pico W. The Pico W communicated wirelessly with nearby Bluetooth-enabled devices, demonstrating basic Bluetooth communication and wireless connectivity concepts.

MicroPython Code

```
from bluetooth import BLE

ble = BLE()
ble.active(True)

print("Bluetooth is ON")
print("Waiting for connection...")

while True:
    pass
```


Experiment: Establish Wireless Communication Between a Raspberry Pi

Aim

To establish wireless communication using a Raspberry Pi through Wi-Fi networking and exchange data between devices.

Apparatus Required

- Raspberry Pi
- Wi-Fi network or hotspot
- MicroSD card with Raspberry Pi OS
- Power adapter
- USB keyboard and mouse
- Monitor with HDMI cable
- Laptop or smartphone for communication testing

Theory

Wireless communication allows devices to exchange data without physical cables. The Raspberry Pi has built-in Wi-Fi capability that enables it to connect to wireless networks and communicate with other devices over the internet or local network.

In this experiment: The Raspberry Pi connects to a Wi-Fi network. Network communication is established using IP addresses. Data can be exchanged wirelessly between the Raspberry Pi and another device using network tools such as ping. Wireless communication is widely used in IoT, automation, robotics, and smart devices.

Procedure

1. Insert the MicroSD card containing Raspberry Pi OS into the Raspberry Pi.
2. Power ON the Raspberry Pi.
3. Open the Wi-Fi settings and connect the Raspberry Pi to a wireless network.
4. Open the terminal window.
5. Check the IP address of the Raspberry Pi using a network command.
6. Use the ping command to test communication with another device on the same network.
7. Observe successful packet transmission and reception.

Working Principle

The Raspberry Pi connects to a wireless router using Wi-Fi. The router assigns an IP address to the Raspberry Pi. Data packets are transmitted wirelessly over the network. Communication between devices is verified using network commands such as ping.

Applications

Internet of Things (IoT) Smart home automation Wireless monitoring systems Robotics Remote data transmission
Industrial automation

Conclusion

The experiment was successfully performed, and wireless communication was established using the Raspberry Pi through Wi-Fi networking. The Raspberry Pi communicated with other devices over the wireless network, demonstrating basic networking and wireless communication concepts.

Linux Commands

```
hostname -I      # Displays Raspberry Pi IP address
ping google.com  # Tests internet connectivity
ping 192.168.1.2 # Tests communication with another device
ifconfig         # Displays network configuration
```

Experiment: Interface a DC Motor with a Raspberry Pi

Aim

To interface a DC motor with a Raspberry Pi and control its operation using GPIO pins through Python programming.

Apparatus Required

- Raspberry Pi
- DC motor
- Motor driver module (L293D or L298N)
- Breadboard
- Jumper wires
- External power supply
- MicroSD card with Raspberry Pi OS
- USB keyboard and mouse
- Power adapter

Theory

A DC motor converts electrical energy into mechanical rotation. Since the Raspberry Pi GPIO pins cannot supply enough current to drive a motor directly, a motor driver IC such as L293D or L298N is used.

In this experiment: The Raspberry Pi sends control signals through GPIO pins. The motor driver receives these signals and powers the DC motor. The motor rotates according to the program instructions. The motor driver protects the Raspberry Pi from high current and voltage generated by the motor.

Procedure

1. Connect the DC motor terminals to the motor driver output pins.
2. Connect the motor driver input pins to GPIO pins 17 and 18 of Raspberry Pi.
3. Connect the motor driver GND to Raspberry Pi GND.
4. Connect the external power supply to the motor driver.
5. Power ON the Raspberry Pi.
6. Open the terminal window.
7. Create a Python file for motor control.
8. Write and save the Python program.
9. Run the program.
10. Observe the rotation of the DC motor.

Working Principle

The Raspberry Pi sends HIGH and LOW signals to the motor driver inputs. The motor driver controls the direction of current flow through the DC motor. Depending on the input signals: The motor rotates forward. The motor rotates backward. The motor stops. The process repeats continuously according to the program.

Applications

Robotics Conveyor systems Home automation Industrial automation Electric vehicles Smart machines

Conclusion

The experiment was successfully performed, and the DC motor was interfaced with the Raspberry Pi using a motor driver module. The Raspberry Pi controlled the motor direction and operation through Python programming, demonstrating motor control and GPIO interfacing techniques.

Python Code

```
import RPi.GPIO as GPIO
import time

IN1 = 17
IN2 = 18

GPIO.setmode(GPIO.BCM)

GPIO.setup(IN1, GPIO.OUT)
GPIO.setup(IN2, GPIO.OUT)

while True:

    # Motor rotates forward
    GPIO.output(IN1, GPIO.HIGH)
    GPIO.output(IN2, GPIO.LOW)
    time.sleep(2)

    # Motor stops
    GPIO.output(IN1, GPIO.LOW)
    GPIO.output(IN2, GPIO.LOW)
    time.sleep(1)

    # Motor rotates backward
    GPIO.output(IN1, GPIO.LOW)
    GPIO.output(IN2, GPIO.HIGH)
    time.sleep(2)

    # Motor stops
    GPIO.output(IN1, GPIO.LOW)
    GPIO.output(IN2, GPIO.LOW)
    time.sleep(1)
```

Experiment: Interface an Ultrasonic Sensor (HC-SR04) with a Raspberry Pi

Aim

To interface an HC-SR04 ultrasonic sensor with a Raspberry Pi and measure distance using Python programming.

Apparatus Required

- Raspberry Pi
- HC-SR04 ultrasonic sensor
- Breadboard
- Jumper wires
- Resistors (1k Ω and 2k Ω for voltage divider)
- MicroSD card with Raspberry Pi OS
- USB keyboard and mouse
- Power adapter

Theory

The HC-SR04 ultrasonic sensor is used to measure distance using ultrasonic sound waves. The sensor consists of: Trigger (TRIG) pin Echo (ECHO) pin Transmitter and receiver modules Working principle: The Raspberry Pi sends a short pulse to the TRIG pin. The sensor emits ultrasonic waves. The waves reflect back after hitting an object. The ECHO pin sends a signal back to the Raspberry Pi. The Raspberry Pi calculates distance based on the time taken for the echo to return. Since the ECHO pin outputs 5V, a voltage divider is used to safely connect it to the Raspberry Pi's 3.3V GPIO pin.

Procedure

1. Connect the VCC pin of HC-SR04 to the 5V pin of Raspberry Pi.
2. Connect the GND pin to the GND of Raspberry Pi.
3. Connect the TRIG pin to GPIO pin 23.
4. Connect the ECHO pin to GPIO pin 24 through a voltage divider circuit.
5. Power ON the Raspberry Pi.
6. Open the terminal window.
7. Create a Python file for the ultrasonic sensor program.
8. Write and save the Python code.
9. Run the program.
10. Place an object in front of the sensor and observe the measured distance.

Working Principle

The Raspberry Pi sends a trigger pulse to the HC-SR04 sensor. The sensor emits ultrasonic waves toward an object. The waves reflect back to the sensor receiver. The ECHO pin returns a signal whose duration depends on the distance. The Raspberry Pi calculates and displays the distance in centimeters.

Applications

Obstacle detection systems Robotics Distance measurement Smart parking systems Industrial automation Water level monitoring

Conclusion

The experiment was successfully performed, and the HC-SR04 ultrasonic sensor was interfaced with the Raspberry Pi using GPIO pins. The Raspberry Pi measured and displayed distance accurately using Python programming, demonstrating ultrasonic sensing and distance measurement techniques.

Python Code

```
import RPi.GPIO as GPIO
import time

TRIG = 23
ECHO = 24

GPIO.setmode(GPIO.BCM)

GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)

GPIO.output(TRIG, False)
time.sleep(2)

while True:

    GPIO.output(TRIG, True)
    time.sleep(0.00001)
    GPIO.output(TRIG, False)

    while GPIO.input(ECHO) == 0:
        pulse_start = time.time()

    while GPIO.input(ECHO) == 1:
        pulse_end = time.time()

    pulse_duration = pulse_end - pulse_start

    distance = pulse_duration * 17150
    distance = round(distance, 2)

    print("Distance:", distance, "cm")

    time.sleep(1)
```

Experiment: Interface a DHT11 Sensor with a Raspberry Pi

Aim

To interface a DHT11 temperature and humidity sensor with a Raspberry Pi and measure temperature and humidity using Python programming.

Apparatus Required

- Raspberry Pi
- DHT11 temperature and humidity sensor
- Breadboard
- Jumper wires
- 10k Ω resistor
- MicroSD card with Raspberry Pi OS
- USB keyboard and mouse
- Power adapter

Theory

The DHT11 is a digital sensor used to measure temperature and humidity. It contains a humidity sensing component and a thermistor for temperature measurement.

The Raspberry Pi communicates with the DHT11 sensor through GPIO pins. The sensor sends digital data representing temperature and humidity values.

In this experiment: The DHT11 sensor is connected to a GPIO pin of the Raspberry Pi. Python libraries are used to read sensor data. The measured temperature and humidity values are displayed on the screen.

Procedure

1. Connect the VCC pin of the DHT11 sensor to the 3.3V pin of Raspberry Pi.
2. Connect the GND pin of the sensor to the GND pin of Raspberry Pi.
3. Connect the DATA pin of the sensor to GPIO pin 4.
4. Connect a 10k Ω pull-up resistor between the DATA pin and VCC.
5. Power ON the Raspberry Pi.
6. Install the required Python library for DHT11.
7. Write the Python program to read sensor data.
8. Run the program.
9. Observe the temperature and humidity values displayed on the screen.

Working Principle

The DHT11 sensor measures surrounding temperature and humidity. The sensor sends digital data to GPIO pin 4 of the Raspberry Pi. The Python program reads and processes the sensor data. The measured values are displayed on the monitor screen.

Applications

Weather monitoring systems Smart home automation IoT environmental monitoring Agricultural monitoring systems
Industrial climate control

Conclusion

The experiment was successfully performed, and the DHT11 sensor was interfaced with the Raspberry Pi using GPIO pins. Temperature and humidity data were measured and displayed correctly using Python programming, demonstrating sensor interfacing and environmental monitoring using Raspberry Pi.

Python Code

```
import Adafruit_DHT

sensor = Adafruit_DHT.DHT11
pin = 4

humidity, temperature = Adafruit_DHT.read_retry(sensor, pin)

if humidity is not None and temperature is not None:
    print("Temperature =", temperature, "°C")
    print("Humidity =", humidity, "%")
else:
    print("Failed to retrieve data")
```


Experiment: Interface a Push Button with a Raspberry Pi

Aim

To interface a push button with a Raspberry Pi and detect button presses using GPIO pins through a Python program.

Apparatus Required

- Raspberry Pi
- Push button switch
- Breadboard
- Jumper wires
- 10k Ω resistor
- MicroSD card with Raspberry Pi OS
- USB keyboard and mouse
- Power adapter

Theory

A push button is an input device used to send signals to electronic systems. The Raspberry Pi uses GPIO (General Purpose Input Output) pins to detect the state of the push button.

In this experiment: One terminal of the push button is connected to a GPIO pin. The GPIO pin reads HIGH or LOW depending on the button state. When the button is pressed, the Raspberry Pi detects the signal and displays a message. A pull-down resistor is used to ensure stable input values and prevent floating signals.

Procedure

1. Connect one terminal of the push button to GPIO pin 17 of the Raspberry Pi.
2. Connect the other terminal of the push button to the 3.3V pin.
3. Connect a 10k Ω pull-down resistor between GPIO pin 17 and GND.
4. Power ON the Raspberry Pi.
5. Open the terminal window.
6. Create a Python file for the push button program.
7. Write the Python code and save the file.
8. Run the program.
9. Press the push button and observe the output on the screen.

Working Principle

GPIO pin 17 is configured as an input pin. The pull-down resistor keeps the pin LOW when the button is not pressed. When the button is pressed, the GPIO pin receives a HIGH signal. The Raspberry Pi detects the signal and prints "Button Pressed" on the screen.

Applications

Home automation systems Security systems User input controls Industrial control systems IoT devices

Conclusion

The experiment was successfully performed, and the push button was interfaced with the Raspberry Pi using GPIO pins. The Raspberry Pi detected the button press correctly through the Python program, demonstrating basic digital input interfacing and GPIO programming.

Python Code

```
import RPi.GPIO as GPIO
import time

BUTTON = 17

GPIO.setmode(GPIO.BCM)
GPIO.setup(BUTTON, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)

while True:
    if GPIO.input(BUTTON) == GPIO.HIGH:
        print("Button Pressed")
        time.sleep(0.2)
```

Experiment: Interface an LED with a Raspberry Pi

Aim

To interface an LED with a Raspberry Pi and control it using GPIO pins through a Python program.

Apparatus Required

- Raspberry Pi
- LED
- 220 Ω resistor
- Breadboard
- Jumper wires
- MicroSD card with Raspberry Pi OS
- USB keyboard and mouse
- Power adapter

Theory

The Raspberry Pi contains GPIO (General Purpose Input Output) pins that allow it to interact with external electronic components such as LEDs, sensors, and motors.

In this experiment: An LED is connected to one GPIO pin of the Raspberry Pi. The GPIO pin is programmed as an OUTPUT pin. The Raspberry Pi sends HIGH and LOW signals to turn the LED ON and OFF. A resistor is used to limit the current flowing through the LED and protect it from damage.

Procedure

1. Connect the longer leg (anode) of the LED to GPIO pin 18 of the Raspberry Pi through a 220 Ω resistor.
2. Connect the shorter leg (cathode) of the LED to the GND pin.
3. Power ON the Raspberry Pi.
4. Open the terminal window.
5. Create a Python file for the LED program.
6. Write the Python code to control the LED.
7. Save and run the program.
8. Observe the LED blinking continuously.

Working Principle

The Raspberry Pi configures GPIO pin 18 as an output pin. A HIGH signal turns the LED ON. A LOW signal turns the LED OFF. The `time.sleep(1)` function creates a delay of 1 second. The process repeats continuously, causing the LED to blink.

Applications

Conclusion

The experiment was successfully performed, and the LED was interfaced with the Raspberry Pi using GPIO pins. The LED blinked according to the Python program, demonstrating basic GPIO control and hardware interfacing using the Raspberry Pi.

Python Code

```
import RPi.GPIO as GPIO
import time

LED = 18

GPIO.setmode(GPIO.BCM)
GPIO.setup(LED, GPIO.OUT)

while True:
    GPIO.output(LED, GPIO.HIGH)    # LED ON
    time.sleep(1)

    GPIO.output(LED, GPIO.LOW)     # LED OFF
    time.sleep(1)
```

Experiment: Gain an Understanding of the Raspberry Pi

Aim

To study and understand the basic features, components, and working of the Raspberry Pi computer.

Apparatus Required

- Raspberry Pi
- MicroSD card
- Power adapter
- HDMI cable
- Monitor
- USB keyboard and mouse
- Internet connection (optional)

Theory

The Raspberry Pi is a small, low-cost single-board computer developed to promote computer science education and electronics projects. It can perform most functions of a normal computer such as browsing the internet, programming, multimedia playback, and hardware interfacing.

Main components of the Raspberry Pi include: Processor (CPU) RAM GPIO (General Purpose Input Output) pins USB ports HDMI port Power supply connector MicroSD card slot The Raspberry Pi uses an operating system, usually Raspberry Pi OS, which is stored on a MicroSD card.

Features of Raspberry Pi

Small and portable Low power consumption Supports programming languages like Python, C, and Java Built-in GPIO pins for electronics interfacing Supports networking and internet access Can be used for IoT, robotics, automation, and multimedia projects

Procedure

1. Insert the MicroSD card containing Raspberry Pi OS into the Raspberry Pi.
2. Connect the monitor using an HDMI cable.
3. Connect the USB keyboard and mouse.
4. Connect the power adapter to turn ON the Raspberry Pi.
5. Wait for the operating system to boot.
6. Observe the desktop interface and basic functions.
7. Identify different ports and GPIO pins on the board.
8. Open the terminal and run simple commands.

Working Principle

The Raspberry Pi works like a compact computer. When power is supplied: The operating system loads from the MicroSD card. The processor executes instructions. Users interact using keyboard, mouse, and monitor. GPIO pins allow communication with external electronic devices and sensors.

Applications of Raspberry Pi

Home automation Robotics IoT projects Media centers Web servers Educational programming Smart surveillance systems

Conclusion

The Raspberry Pi was studied successfully, and its basic hardware components, features, and working principles were understood. The experiment helped in learning about single-board computers and their applications in electronics, programming, and automation projects.

Basic Raspberry Pi Commands

```
pwd          # Shows current directory
ls           # Lists files and folders
cd           # Changes directory
mkdir test   # Creates a folder named test
date         # Displays current date and time
```

Experiment: Blink Multiple LEDs with Arduino

Aim

To interface multiple LEDs with an Arduino UNO and make them blink one after another using Arduino programming.

Apparatus Required

- Arduino UNO
- 3 LEDs
- 3 resistors (220Ω each)
- Jumper wires
- USB cable
- Breadboard

Theory

LEDs (Light Emitting Diodes) emit light when electric current passes through them. The Arduino UNO controls the LEDs through its digital output pins.

In this experiment: Three LEDs are connected to digital pins 13, 12, and 11. The Arduino sends HIGH and LOW signals to each pin. The LEDs blink in sequence, creating a running light effect. This demonstrates how multiple output devices can be controlled using a microcontroller.

Procedure

1. Connect the positive leg (anode) of the first LED to pin 13 through a 220Ω resistor.
2. Connect the second LED to pin 12 through another resistor.
3. Connect the third LED to pin 11 through another resistor.
4. Connect all negative legs (cathodes) of the LEDs to the GND pin of Arduino.
5. Connect the Arduino UNO to the computer using a USB cable.
6. Open Arduino IDE and write the program.
7. Select the correct board and COM port.
8. Upload the code to the Arduino UNO.
9. Observe the LEDs blinking one after another.

Working Principle

The Arduino turns ON the first LED and waits for 0.5 seconds. Then it turns OFF the first LED and turns ON the second LED. The same process repeats for the third LED. The loop() function repeats continuously, creating a sequential blinking effect.

Conclusion

The experiment was successfully performed, and multiple LEDs blinked sequentially using the Arduino UNO. This experiment demonstrates the control of multiple digital output devices using a microcontroller and helps in understanding basic Arduino programming and interfacing concepts.

Arduino Code

```
// Multiple LED Blink using Arduino UNO

int led1 = 13;
int led2 = 12;
int led3 = 11;

void setup() {
  pinMode(led1, OUTPUT);
  pinMode(led2, OUTPUT);
  pinMode(led3, OUTPUT);
}

void loop() {

  digitalWrite(led1, HIGH);
  delay(500);
  digitalWrite(led1, LOW);

  digitalWrite(led2, HIGH);
  delay(500);
  digitalWrite(led2, LOW);

  digitalWrite(led3, HIGH);
  delay(500);
  digitalWrite(led3, LOW);
}
```


Experiment: Blink an LED with Arduino

Aim

To interface an LED with an Arduino UNO and make it blink continuously using a simple Arduino program.

Apparatus Required

- Arduino UNO
- LED
- 220 Ω resistor
- Jumper wires
- USB cable
- Breadboard (optional)

Theory

An LED (Light Emitting Diode) is an electronic component that emits light when electric current flows through it. The Arduino UNO can control the flow of current through its digital output pins.

In this experiment: The LED is connected to digital pin 13. The Arduino sends a HIGH signal to turn the LED ON. It sends a LOW signal to turn the LED OFF. A delay is added between ON and OFF states to create a blinking effect. The blinking process repeats continuously inside the loop() function.

Procedure

1. Connect the longer leg (anode) of the LED to digital pin 13 through a 220 Ω resistor.
2. Connect the shorter leg (cathode) of the LED to the GND pin of Arduino.
3. Connect the Arduino UNO to the computer using a USB cable.
4. Open the Arduino IDE on the computer.
5. Write or paste the blink program into the IDE.
6. Select the correct board and COM port.
7. Upload the code to the Arduino UNO.
8. Observe the LED blinking repeatedly.

Working Principle

The Arduino continuously executes the loop() function: digitalWrite(13, HIGH) turns the LED ON. delay(1000) keeps it ON for 1 second. digitalWrite(13, LOW) turns the LED OFF. Another delay keeps it OFF for 1 second. This creates a blinking effect.

Conclusion

The experiment was successfully performed, and the LED blinked continuously using the Arduino UNO. This experiment demonstrates basic Arduino programming, digital output control, and interfacing of electronic components

with a microcontroller.

Arduino Code

```
// Blink LED using Arduino UNO

void setup() {
  pinMode(13, OUTPUT);  // Set pin 13 as output
}

void loop() {
  digitalWrite(13, HIGH); // LED ON
  delay(1000);           // Wait for 1 second

  digitalWrite(13, LOW);  // LED OFF
  delay(1000);           // Wait for 1 second
}
```